

ASSIGNMENT-9.1

2303A52249

A.VAMSHI KRISHNA

TASK-1:

Consider the following Python function: `def find_max(numbers): return max(numbers)`

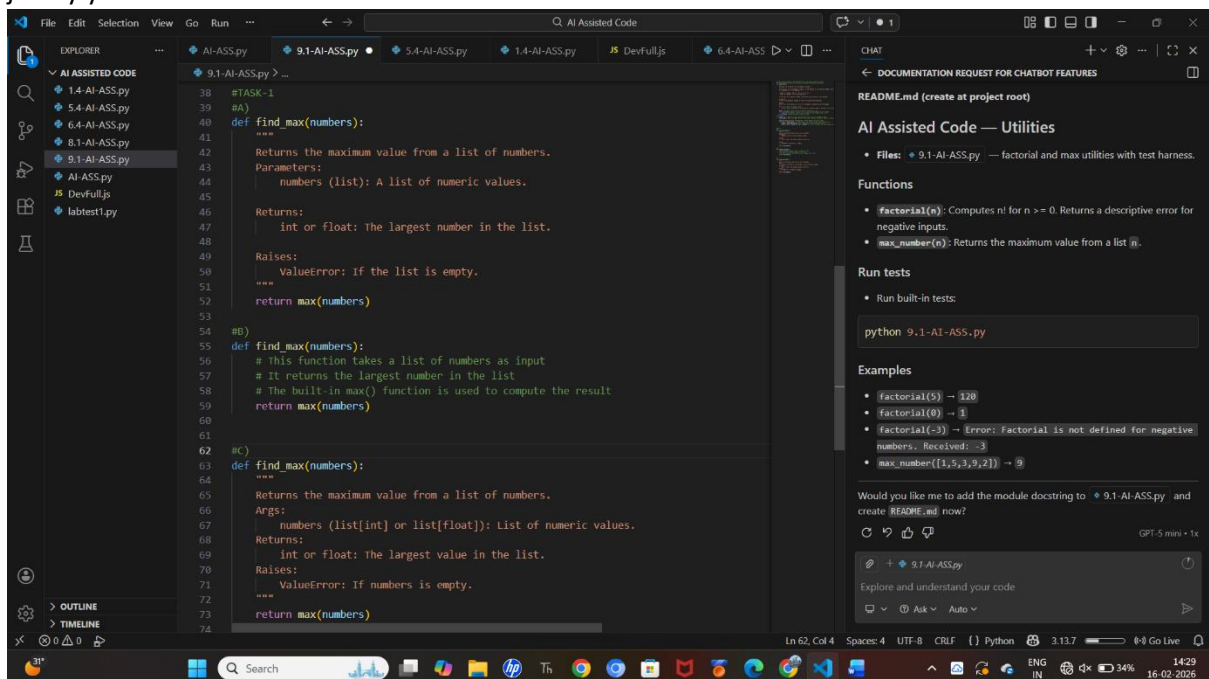
Prompt: Write documentation for the function in all three formats:

(a) Docstring

(b) Inline comments

(c) Google-style documentation

- Critically compare the three approaches. Discuss the advantages, disadvantages, and suitable use cases of each style.
- Recommend which documentation style is most effective for a mathematical utilities library and justify your answer.



Docstring (Standard Style):

Advantages

- Accessible via `help(find_max)`
- Good for small projects
- Follows Python conventions (PEP 257)

Disadvantages

- Less structured than Google style
- Not always consistent formatting

Suitable Use Case

- Small academic projects
 - Simple scripts
 - Beginner-level coding
-

Inline Comments:

Advantages

- Easy to write
- Helpful for explaining logic line-by-line
- Good for beginners

Disadvantages

- Not accessible via documentation tools
- Can become messy in large programs
- Harder to maintain

Suitable Use Case

- Explaining complex logic
- Debugging
- Quick clarification inside code

Google-Style Documentation:

Advantages

- Clear structured format (Args, Returns, Raises)
- Used in large-scale projects
- Compatible with tools like Sphinx
- Easy to read and maintain

Disadvantages

- Slightly longer to write
- Requires formatting discipline

Suitable Use Case

- Professional software development
- Libraries and APIs
- Team projects

TASK-2:

Prompt: Consider the following Python function:

```
def login(user, password, credentials):  
    return credentials.get(user) == password
```

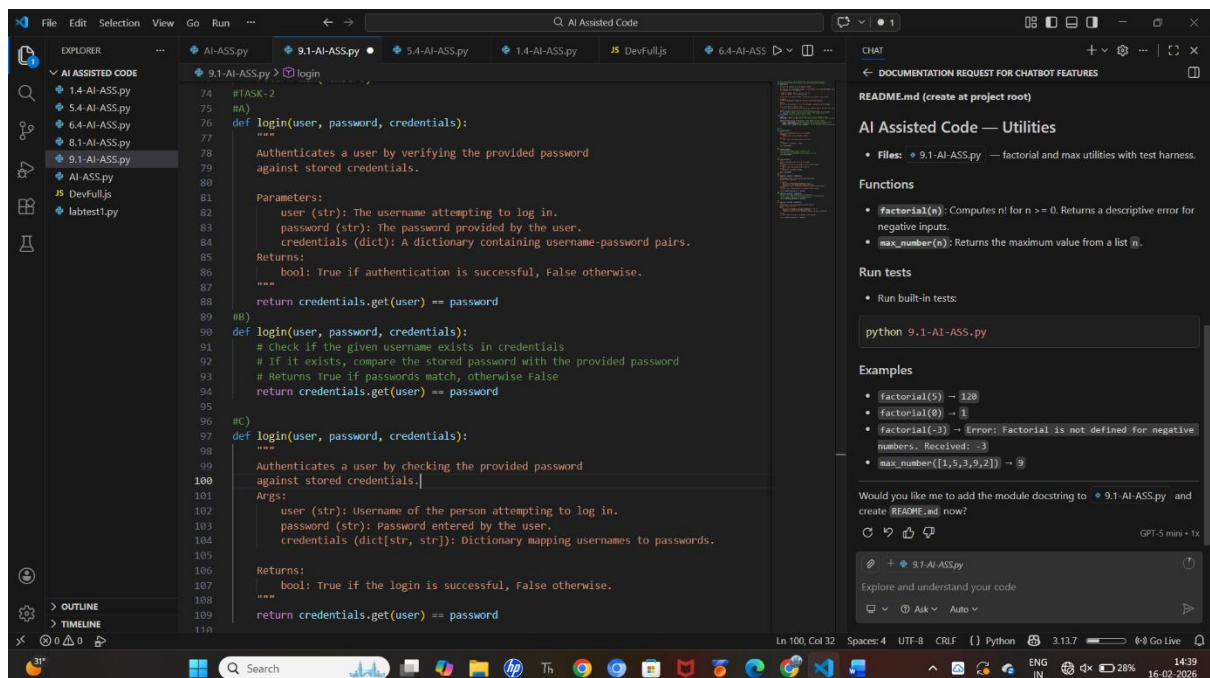
Task:

1. Write documentation in all three formats.
2. Critically compare the approaches.
3. Recommend which style would be most helpful for new developers onboarding a project, and justify your choice.

(a) Standard Docstring

(b) Inline Comments

(c) Google-Style Docstring



Standard Docstring:

Advantages

- Clean and simple
- Easily accessible using `help(login)`
- Suitable for small applications

Disadvantages

- Formatting may vary between developers
- Slightly less structured than Google style

2. Inline Comments:

Advantages

- Very easy to understand for beginners
- Good for explaining logic inside complex code

Disadvantages

- Cannot be extracted by documentation tools
- Not suitable for large projects
- Does not describe parameters clearly

3. Google-Style Docstring:

Advantages

- Clearly separates Args and Returns
- Industry-standard format
- Easy for IDEs and documentation generators
- Very readable for teams

Disadvantages

- Slightly longer to write
- Requires consistent formatting

TASK-3:

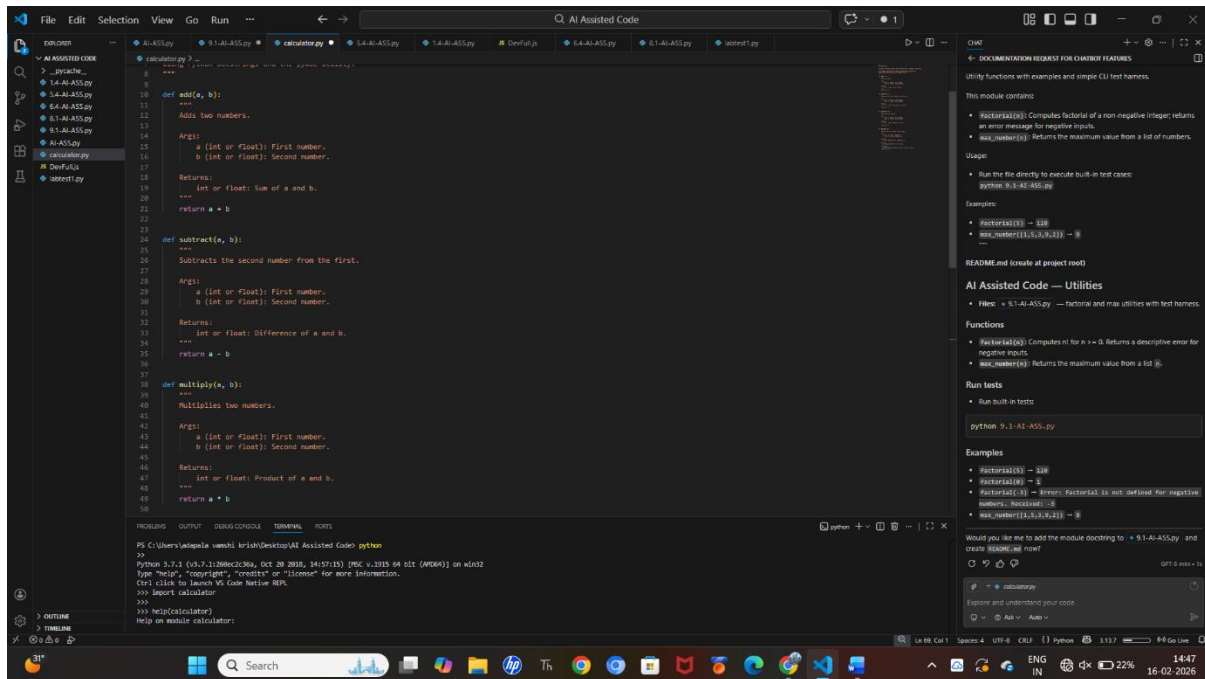
Prompt: Calculator (Automatic Documentation Generation)

Task: Design a Python module named calculator.py and demonstrate automatic documentation generation.

Instructions:

1. Create a Python module calculator.py that includes the following functions, each written with appropriate docstrings:
 - o add(a, b) – returns the sum of two numbers
 - o subtract(a, b) – returns the difference of two numbers
 - o multiply(a, b) – returns the product of two numbers
 - o divide(a, b) – returns the quotient of two numbers
2. Display the module documentation in the terminal using Python's documentation tools.
3. Generate and export the module documentation in HTML format using the pydoc utility, and open the generated HTML file in a web browser to verify the output.

CODE:



```
def add(a, b):
    """
    Adds two numbers.

    Args:
        a (int or float): First number.
        b (int or float): Second number.

    Returns:
        int or float: Sum of a and b.
    """
    return a + b

def subtract(a, b):
    """
    Subtracts the second number from the first.

    Args:
        a (int or float): First number.
        b (int or float): Second number.

    Returns:
        int or float: Difference of a and b.
    """
    return a - b

def multiply(a, b):
    """
    Multiplies two numbers.

    Args:
        a (int or float): First number.
        b (int or float): Second number.

    Returns:
        int or float: Product of a and b.
    """
    return a * b
```

Documentation for calculator module:

- Utility Functions with examples and simple CLI test harness.
- This module contains:
 - `factorial(n)`: Computes factorial of a non-negative integer; returns an error message for negative inputs.
 - `max_number(s)`: Returns the maximum value from a list of numbers.
- Usage:
 - Run the file directly to execute built-in test cases: `python 9-1-AI-ASS.py`
- Examples:
 - `factorial(5) → 120`
 - `max_number([1,5,3,9,11]) → 11`

Observation:

During this experiment, a Python module named `calculator.py` was created with properly written docstrings for each function. The module documentation and function-level documentation were automatically generated using Python's built-in documentation tool (pydoc).

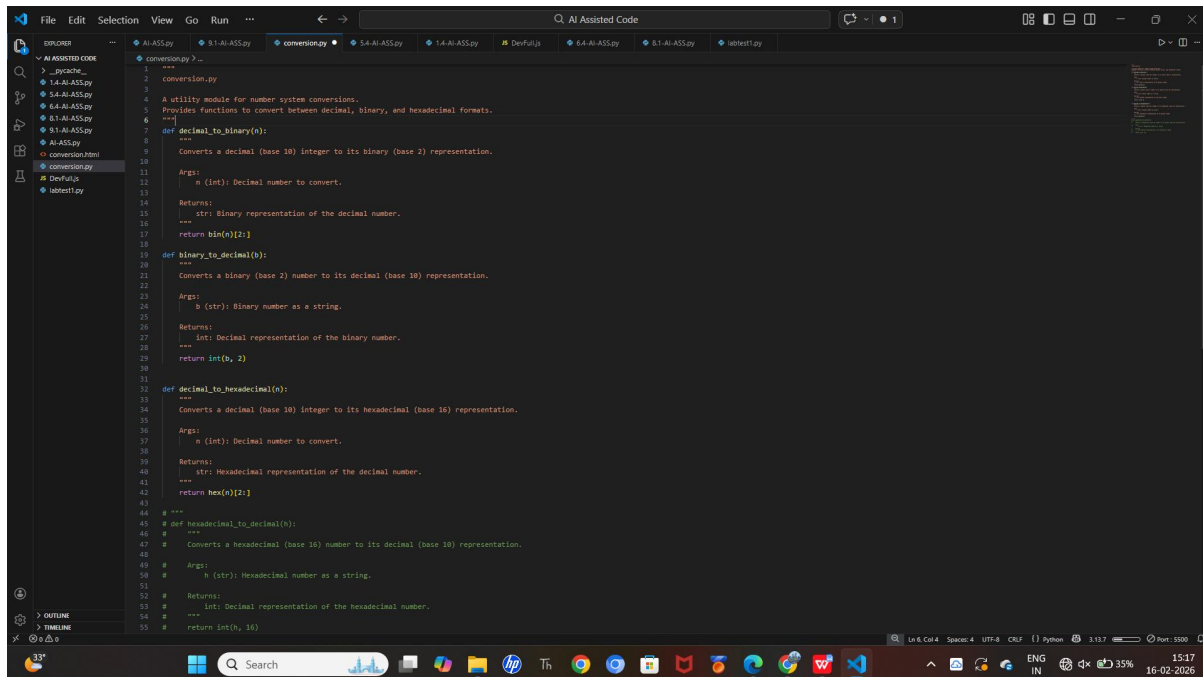
TASK-4:

Prompt: Conversion Utilities Module

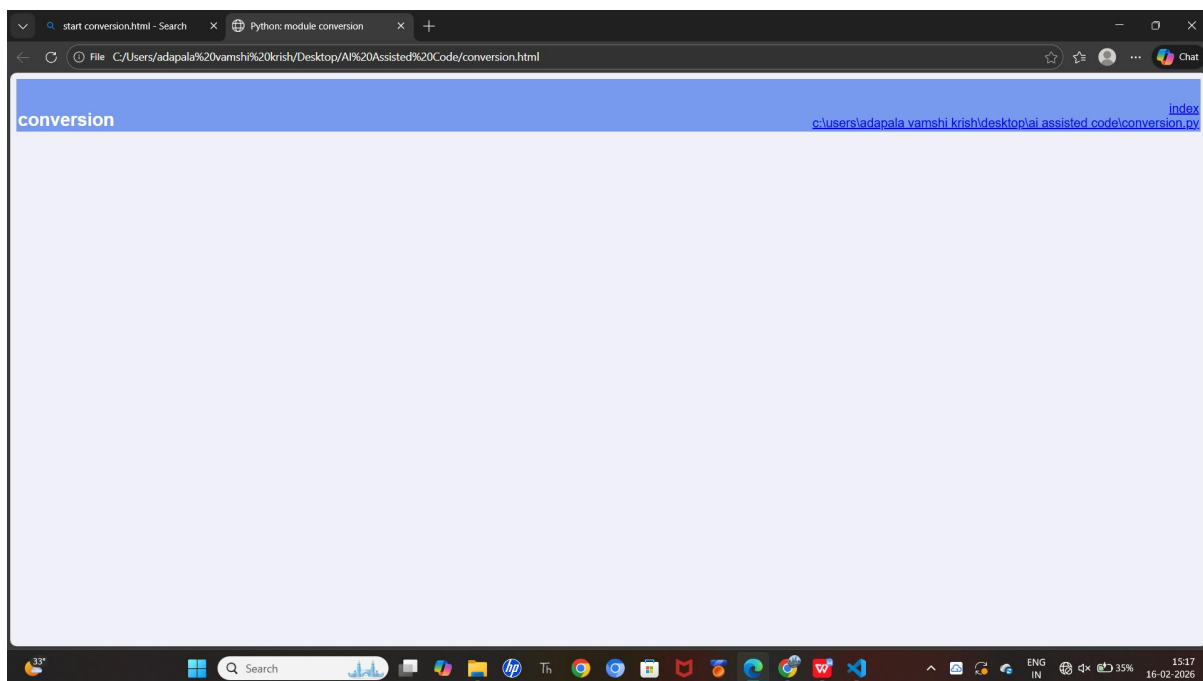
Task:

1. Write a module named `conversion.py` with functions:
 - o `decimal_to_binary(n)`
 - o `binary_to_decimal(b)`
 - o `decimal_to_hexadecimal(n)`
2. Use Copilot for auto-generating docstrings.
3. Generate documentation in the terminal.
4. Export the documentation in HTML format and open it in a browser.

CODE:



```
1 """
2 conversion.py
3
4 A utility module for number system conversions.
5 Provides functions to convert between decimal, binary, and hexadecimal formats.
6 """
7 def decimal_to_binary(n):
8     """
9     Converts a decimal (base 10) integer to its binary (base 2) representation.
10
11     Args:
12         n (int): Decimal number to convert.
13
14     Returns:
15         str: Binary representation of the decimal number.
16     """
17     return bin(n)[2:]
18
19 def binary_to_decimal(b):
20     """
21     Converts a binary (base 2) number to its decimal (base 10) representation.
22
23     Args:
24         b (str): Binary number as a string.
25
26     Returns:
27         int: Decimal representation of the binary number.
28     """
29     return int(b, 2)
30
31 def decimal_to_hexadecimal(n):
32     """
33     Converts a decimal (base 10) integer to its hexadecimal (base 16) representation.
34
35     Args:
36         n (int): Decimal number to convert.
37
38     Returns:
39         str: Hexadecimal representation of the decimal number.
40     """
41     return hex(n)[2:]
42
43 # """
44 # def hexadecimal_to_decimal(h):
45 #     """
46 #     Converts a hexadecimal (base 16) number to its decimal (base 10) representation.
47 #
48 #     Args:
49 #         h (str): Hexadecimal number as a string.
50 #
51 #     Returns:
52 #         int: Decimal representation of the hexadecimal number.
53 #     """
54 #     return int(h, 16)
```



Observation :

The conversion.py module successfully demonstrated automatic documentation generation using Python's pydoc tool. The AI-assisted docstrings improved clarity and structure. The documentation was easily exported to HTML format, making it suitable for professional use and sharing.

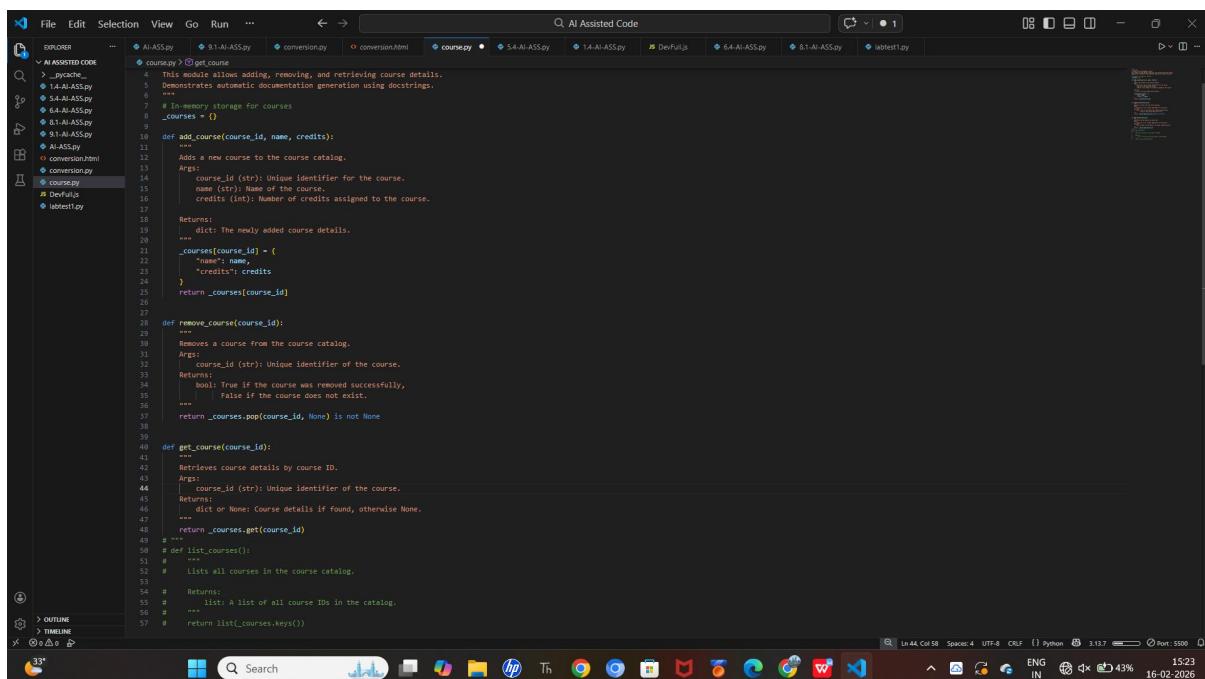
TASK-5:

Prompt: Course Management Module

Task:

1. Create a module `course.py` with functions:
 - o `add_course(course_id, name, credits)`
 - o `remove_course(course_id)`
 - o `get_course(course_id)`
2. Add docstrings with Copilot.
3. Generate documentation in the terminal.
4. Export the documentation in HTML format and open it in a browser.

CODE:



```
4 This module allows adding, removing, and retrieving course details.
5 Demonstrates automatic documentation generation using docstrings.
6 """
7 # In-memory storage for courses
8 _courses = {}
9
10 def add_course(course_id, name, credits):
11     """
12     Adds a new course to the course catalog.
13     Args:
14         course_id (str): Unique Identifier for the course.
15         name (str): Name of the course.
16         credits (int): Number of credits assigned to the course.
17     Returns:
18         dict: The newly added course details.
19     """
20     _courses[course_id] = {
21         "name": name,
22         "credits": credits
23     }
24     return _courses[course_id]
25
26 def remove_course(course_id):
27     """
28     Removes a course from the course catalog.
29     Args:
30         course_id (str): Unique Identifier of the course.
31     Returns:
32         bool: True if the course was removed successfully,
33             False if the course does not exist.
34     """
35     return _courses.pop(course_id, None) is not None
36
37 def get_course(course_id):
38     """
39     Retrieves course details by course ID.
40     Args:
41         course_id (str): Unique Identifier of the course.
42     Returns:
43         dict or None: Course details if found, otherwise None.
44     """
45     return _courses.get(course_id)
46
47 # def list_courses():
48 #     """
49 #     Lists all courses in the course catalog.
50 #     Returns:
51 #         list: A list of all course IDs in the catalog.
52 #     """
53 #     return list(_courses.keys())
```

Observation:

The `course.py` module successfully demonstrated automatic documentation generation. The docstrings written using Copilot were extracted by `pydoc` and displayed both in the terminal and HTML format. This experiment highlights the importance of structured documentation for maintainability and onboarding new developers.